

Technical Overview of the DeBERTa-Based Phishing Email Detection System

1. DeBERTa-v3-large Model Implementation

The final transformer architecture selected for the project was DeBERTa-v3-large. The selection followed experimentation with DistilBERT, RoBERTa, DeBERTa-v3 and ModernBERT. DeBERTa-v3 demonstrated the strongest observed performance among the evaluated architectures and was therefore developed further for the final system.

The model was fine-tuned for binary classification of Safe Email and Phishing Email. The implementation used a maximum sequence length of 512 tokens, three training epochs, AdamW optimization, a learning rate of 1e-5, weight decay of 0.01 and validation F1 for best-model selection. The final standard DeBERTa-v3-large test evaluation achieved 99.3721% accuracy and 99.1609% F1-score, with seven false positives and four false negatives.

In the deployed workflow, the transformer provides a contextual representation of the email. This allows the classifier to model the meaning and relationships between words in context rather than relying only on fixed keywords or manually defined rules.

2. 12 Feature Extraction

To complement the contextual transformer representation, the project engineered 12 NLP/security features. These features represent explicit linguistic and structural characteristics that may be useful in phishing analysis.

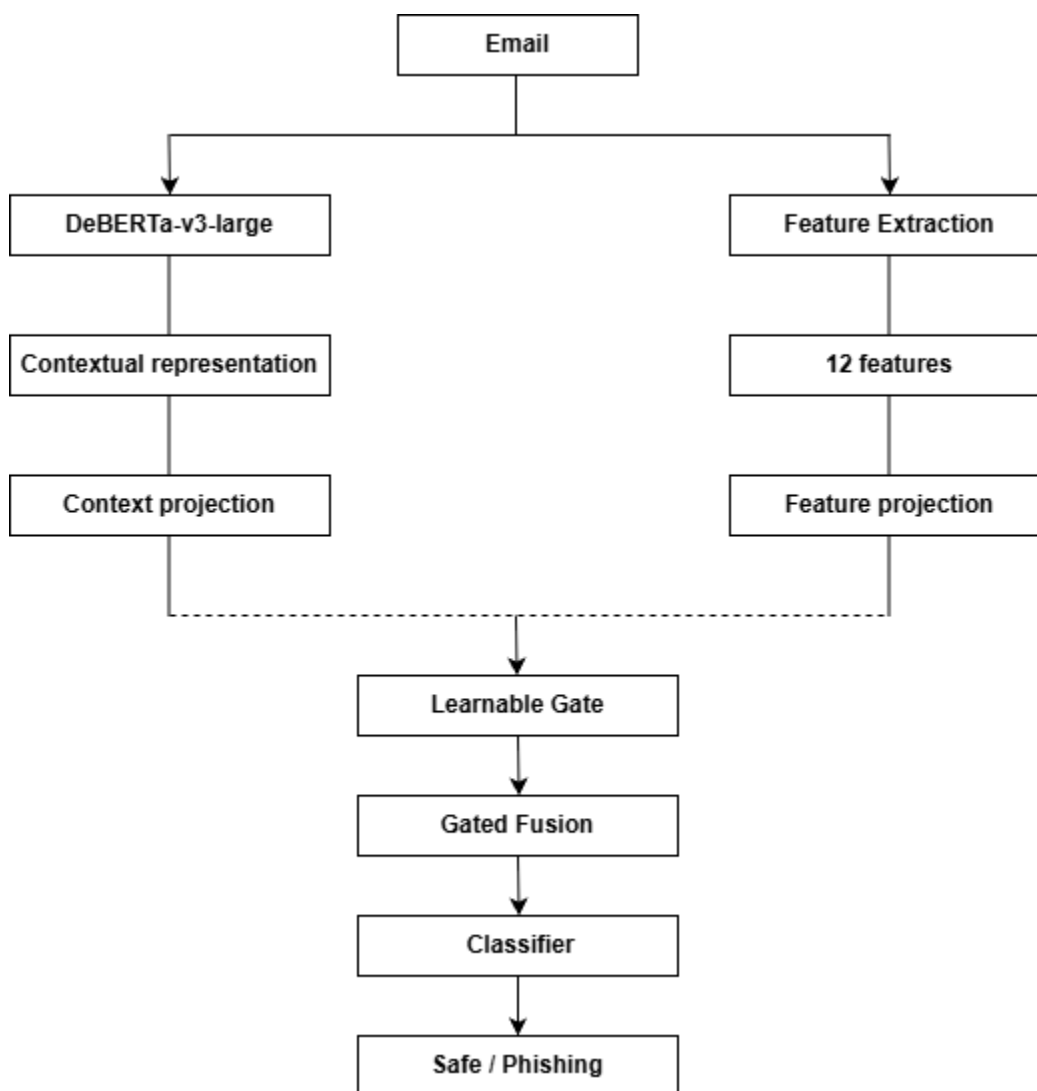
SN	Feature	What it measures	Why relevant to phishing
1	Urgency-word frequency	Occurrence/frequency of urgency-related language	Phishing commonly pressures recipients to act quickly
2	Credential-related terms	Words associated with accounts, passwords, login/verification, etc.	Credential theft is a common phishing objective
3	Threat-related terms	Threatening/warning language	Social-engineering messages may threaten account restriction or other consequences
4	Financial terms	Finance/payment-related language	Helps capture financial-themed phishing

5	Call-to-action terms	Language asking the recipient to perform an action	Phishing generally attempts to make the recipient click, verify, respond, update, etc.
6	URL count	Number of URLs detected in the email	Links can be an important structural phishing indicator
7	Email-address count	Number of email addresses appearing in the text	Provides structural/contextual information
8	Phone-number count	Number of phone numbers detected	Useful for identifying messages directing users toward external contact
9	Exclamation-mark count	Number of ! characters	Can capture emphatic/urgent writing patterns
10	Question-mark count	Number of ? characters	Additional structural/stylistic signal
11	Uppercase-word ratio	Proportion of words written in uppercase	Captures emphasis patterns
12	Logarithmic text length	Log-transformed length of the email	Gives the model information about message length while reducing the effect of very large values

The feature values were normalised using training-set statistics before being supplied to the hybrid feature branch. They are supplementary model inputs rather than hard-coded phishing rules; the final decision remains learned from data.

3. Gated-Hybrid DeBERTa Implementation

The gated-hybrid experiment was designed to investigate whether explicit phishing-related features could complement the contextual information learned by DeBERTa. The architecture contains two parallel information paths: a DeBERTa context branch and an engineered-feature branch.



The architecture has two information paths, that's why we call it **hybrid**: it combines a deep transformer representation with manually engineered phishing/security signals.

And it's gated because the two representations aren't simply concatenated without control. A learnable gating mechanism is used during fusion.

Projection layers transform the contextual and engineered-feature representations into compatible learned spaces. A learnable gate then controls how the two information sources

are fused before the final classifier. This is preferable to assigning fixed manual weights because the fusion behaviour is learned during training.

The hybrid experiment did not improve overall accuracy or F1-score over standard DeBERTa-v3-large. Both achieved 99.3721% accuracy. However, the hybrid increased precision to 99.2343% and reduced false positives from seven to five, while standard DeBERTa retained slightly higher recall and fewer false negatives. The result therefore demonstrates a change in error trade-off rather than a universal improvement.

4. FastAPI Integration

FastAPI provides a backend service that separates machine-learning inference from the user interfaces. The /predict endpoint accepts email text and returns the DeBERTa-based classification result. Supplementary contextual/security indicators can also be returned for user interpretation, but these indicators do not override the model prediction.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "sender": "account-notice@example.com",
    "subject": "Action required for your account",
    "body": "We noticed unusual activity associated with your account. To avoid interruption of access, please review and confirm your account details as soon as possible. If you believe this message was sent i
  }'
```

Request URL

http://127.0.0.1:8000/predict

Server response

Code

Details

200

Response body

```
{
  "sender": "account-notice@example.com",
  "subject": "Action required for your account",
  "prediction": "Phishing Email",
  "confidence": 0.8454366326332092,
  "confidence_percentage": 84.54,
  "safe_probability": 0.1545634677375793,
  "safe_probability_percentage": 15.46,
  "phishing_probability": 0.8454366326332092,
  "phishing_probability_percentage": 84.54,
  "indicators": []
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: http://127.0.0.1:8000
content-length: 249
content-type: application/json
date: Fri, 11 Sep 2026 13:03:04 GMT
server: uvicorn
vary: Origin
```

Responses

Code

Description

Links

A separate /explain endpoint generates SHAP-based explanation data. Separating prediction from explanation is important because ordinary inference is comparatively fast, whereas SHAP requires repeated model evaluations and is substantially more computationally expensive. This API design allows Streamlit and the browser extension to use the same backend without loading the transformer model independently in each interface.

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/explain' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "sender": "security@example.edu",
    "subject": "Account security notification",
    "body": "This is a test security notification from the university IT help desk. Please review your account security settings through the official university portal. If you did not request this notification,
  }'
```

Request URL

http://127.0.0.1:8000/explain

Server response

Code Details

200

Response body

```
{
  "success": true,
  "prediction": "Safe Email",
  "prediction_index": 0,
  "confidence": 0.9996974468231201,
  "confidence_percentage": 99.97,
  "supporting_factors": [
    {
      "token": ":",
      "contribution": 0.135342
    },
    {
      "token": "university",
      "contribution": 0.123603
    },
    {
      "token": ".",
      "contribution": 0.11417
    },
    {
      "token": ":",
      "contribution": 0.098855
    },
    {
      "token": "IT",
      "contribution": 0.092376
    },
    {
      "token": "This"
    }
  ]
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: http://127.0.0.1:8000
content-length: 1520
content-type: application/json
date: Fri, 11 Sep 2026 12:56:30 GMT
server: uvicorn
vary: origin
```

Responses

Code Description

Links

5. SHAP Implementation and Working Output

LIME and SHAP were investigated as post-hoc Explainable AI approaches. SHAP was selected and integrated to provide token-level contribution information indicating which parts of an email influenced the model output.

During integration, unrestricted SHAP evaluation with DeBERTa-v3-large produced substantial computational overhead, long processing times and session instability. The explanation pipeline was therefore optimized independently of normal classification: the classifier retained its full 512-token configuration, while the SHAP workload used a shorter explanation input and a controlled evaluation budget. This preserved normal prediction behavior while making the explanation process stable enough for the prototype.

The working SHAP output is presented to the user as interpretable token-level contributions. It is explanatory rather than a second classifier: the Safe/Phishing decision is produced by the trained model, while SHAP helps communicate which text elements contributed to that decision.

6. Streamlit Working Prediction

Streamlit implements the primary user-facing prototype. A user enters or pastes an email, after which the frontend sends the content to the FastAPI backend. The returned classification is displayed as Safe Email or Phishing Email together with supporting security/context indicators. Explainability is invoked separately so that normal prediction remains responsive and SHAP is generated only when requested.

User email → Streamlit → FastAPI /predict → DeBERTa-based model → prediction → Streamlit result

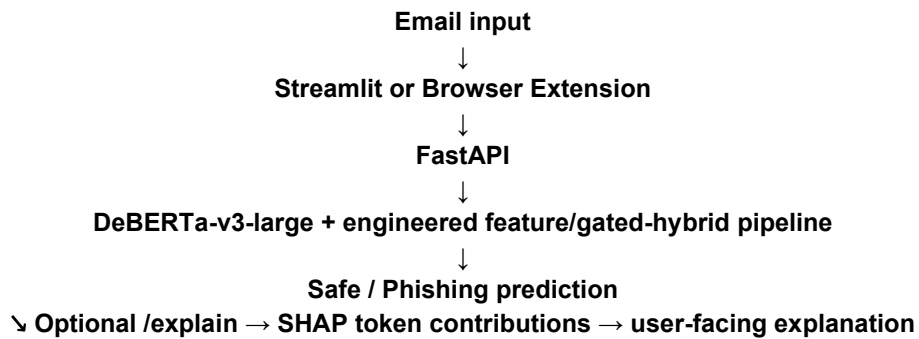
7. Browser-Extension Working Prediction

The browser extension was developed as an additional stretch goal rather than a core requirement of the original proposal. It extends the same detection workflow beyond the Streamlit interface. Email content captured through the extension is sent to the FastAPI backend, which performs model inference and returns the result for display in the extension.

Browser extension → FastAPI backend → DeBERTa-based prediction → Safe / Phishing result

Because both Streamlit and the extension consume the same backend service, the system maintains a modular architecture: model logic and XAI remain centralised in FastAPI while different user interfaces can access the same prediction service.

8. End-to-End Technical Workflow



Overall, the technical progression demonstrates movement from standalone transformer experimentation to a modular, evaluated and explainable phishing-detection prototype. The model-development artefacts establish classification performance, the gated-hybrid experiment demonstrates architectural refinement, FastAPI provides reusable backend integration, SHAP adds interpretability, and Streamlit and the browser extension demonstrate working end-user access.